주요 컴포넌트 상세 분석

소개

이 문서는 IPFS 노드의 주요 컴포넌트들을 상세히 분석하고, 각 컴포넌트의 구조, 책임, 그리고 상호작용 패턴을 설명합니다. IPFS의 모듈화된 아키텍처에서 각 컴포넌트가 어떻게 협력하여 분산 파일 시스템을 구현하는지 이해할 수 있습니다.

1. 신원 및 보안 컴포넌트 (Identity & Security)

1.1 Identity 관리

PeerID (peer.ID)

```
type IpfsNode struct {
Identity peer.ID // 노드의 고유 식별자
}
```

역할과 책임:

- 네트워크에서 노드를 고유하게 식별
- 공개키의 해시값으로 생성
- 모든 네트워크 통신에서 노드 식별자로 사용

생성 과정:

1. 개인키 생성 (RSA, Ed25519, ECDSA 등)
2. 공개키 추출

3. 공개키를 해시하여 PeerID 생성
4. Base58 인코딩으로 문자열 표현

PrivateKey (ic.PrivKey)

```
type IpfsNode struct {
PrivateKey ic.PrivKey `optional:"true"`
}
```

역할과 책임:

- 디지털 서명 생성
- 암호화된 통신 설정
- 노드 신원 증명
- IPNS 레코드 서명

보안 고려사항:

- 안전한 저장소에 암호화되어 저장
- 메모리에서 사용 후 즉시 정리
- 키 회전 및 백업 전략 필요

Peerstore (pstore.Peerstore)

```
type IpfsNode struct {
Peerstore pstore.Peerstore `optional:"true"`
}
```

역할과 책임:

- 다른 피어들의 정보 저장 및 관리
- 피어의 주소, 공개키, 프로토콜 정보 캐싱
- 연결 품질 및 신뢰도 정보 유지

데이터 구조:

- **AddrBook**: 피어 주소 정보
- **KeyBook**: 피어 공개키 정보
- **ProtoBook**: 피어가 지원하는 프로토콜 정보
- **PeerMetadata**: 추가 메타데이터

2. 저장소 계층 컴포넌트 (Storage Layer)

2.1 Blockstore 시스템

GCBlockstore (bstore.GCBlockstore)

```
type IpfsNode struct {
Blockstore bstore.GCBlockstore // 블록 저장소
BaseBlocks node.BaseBlocks // 기본 블록스토어
GCLocker bstore.GCLocker // GC 보호 락
}
```

계층 구조:

```
GCBlockstore (GC 지원)
↓
CachedBlockstore (캐싱)
```

```
↓
BaseBlockstore (실제 저장소)
↓
Datastore Backend (Badger/LevelDB/Flatfs)
```

주요 기능:

- **블록 저장/검색**: CID를 키로 하는 블록 데이터 관리
- **가비지 컬렉션**: 참조되지 않는 블록 정리
- **캐싱**: 자주 사용되는 블록 메모리 캐싱
- **무결성 검증**: 블록 데이터와 CID 일치성 확인

Filestore (filestore.Filestore)

```go
type IpfsNode struct {
Filestore *filestore.Filestore `optional:"true"`
}
```

특별한 기능:

- 파일 시스템의 기존 파일을 IPFS 블록으로 참조
- 데이터 중복 없이 대용량 파일 처리
- 파일 이동/삭제 시 참조 무결성 관리

2.2 Pinning 시스템

Pinner (pin.Pinner)

```
type IpfsNode struct {
Pinning pin.Pinner // 핀 관리자
}
```

핀 타입:

- **Direct Pin**: 특정 블록을 직접 고정
- **Recursive Pin**: 객체와 모든 하위 객체 고정
- **Indirect Pin**: 다른 핀에 의해 간접적으로 고정

데이터 흐름:

```
Pin Request → Pinner → Blockstore → Datastore
↓
Pin Index Update
```

3. 네트워크 계층 컴포넌트 (Network Layer)

3.1 P2P Host (libp2p)

Host (p2phost.Host)

```
type IpfsNode struct {
PeerHost p2phost.Host `optional:"true"`
}
```

핵심 기능:

- **연결 관리**: 피어 간 연결 설정/유지/종료
- **프로토콜 멀티플렉싱**: 여러 프로토콜 동시 지원
- **스트림 관리**: 양방향 통신 스트림 생성/관리
- **주소 관리**: 멀티어드레스 기반 주소 체계

아키텍처:

```
Host
├── Connection Manager (연결 관리)
├── Stream Multiplexer (스트림 멀티플렉싱)
├── Transport Layer (전송 계층)
| ├── TCP Transport
| ├── WebSocket Transport
| └── QUIC Transport
└── Security Layer (보안 계층)
├── TLS
└── Noise Protocol
```

3.2 DHT (Distributed Hash Table)

DHT (*ddht.DHT)

```go
type IpfsNode struct {
DHT *ddht.DHT `optional:"true"`
DHTClient routing.Routing `name:"dhtc" optional:"true"`
```

```
}
```

이중 DHT 구조:

- **WAN DHT**: 공개 인터넷 DHT
- **LAN DHT**: 로컬 네트워크 DHT

주요 기능:

- **피어 라우팅**: 특정 피어의 주소 찾기
- **콘텐츠 라우팅**: 콘텐츠를 제공하는 피어 찾기
- **값 저장**: 키-값 쌍 분산 저장
- **네트워크 부트스트랩**: 새로운 피어의 네트워크 참여

3.3 Bitswap 프로토콜

Bitswap (*bitswap.Bitswap)

```
type IpfsNode struct {
Bitswap *bitswap.Bitswap `optional:"true"`
Exchange exchange.Interface // 블록 교환 인터페이스
}
```

핵심 메커니즘:

- **Want-list**: 필요한 블록 목록 관리
- **Have-list**: 보유한 블록 목록 광고
- **블록 교환**: 피어 간 블록 요청/응답
- **전략 엔진**: 교환 우선순위 및 정책 관리

데이터 흐름:

```
Block Request → Want-list → DHT Query → Peer Discovery
↓
Block Response ← Bitswap Protocol ← Remote Peer
```

4. 서비스 계층 컴포넌트 (Service Layer)

4.1 DAG Service

DAGService (ipld.DAGService)

```go
type IpfsNode struct {
DAG ipld.DAGService // 머클 DAG 서비스
}
```

핵심 기능:

- **노드 추가/검색**: IPLD 노드 저장 및 검색
- **링크 해석**: 노드 간 연결 관계 처리
- **경로 탐색**: IPLD 경로를 통한 데이터 접근
- **배치 처리**: 여러 노드 동시 처리

IPLD 데이터 모델:

```
IPLD Node
├── Data (바이트 데이터)
```

```
└── Links (다른 노드로의 링크)
├── Name (링크 이름)
├── Hash (대상 노드 CID)
└── Size (대상 노드 크기)
```

4.2 Block Service

BlockService (bserv.BlockService)

```
type IpfsNode struct {
Blocks bserv.BlockService // 블록 서비스
}
```

역할:

- Blockstore와 Exchange 계층 연결
- 로컬 블록 검색 후 원격 검색 시도
- 블록 캐싱 및 최적화

검색 흐름:

```
Block Request → BlockService → Blockstore (로컬)
↓ (없으면)
Exchange (Bitswap) → Network
```

4.3 Name System (IPNS)

NameSystem (namesys.NameSystem)

```
type IpfsNode struct {
Namesys namesys.NameSystem // 이름 시스템
}
```

IPNS 레코드 구조:

- **Value**: 가리키는 IPFS 경로
- **Validity**: 레코드 유효 기간
- **Sequence**: 버전 번호
- **TTL**: 캐시 수명
- **Signature**: 디지털 서명

해석 과정:

```
IPNS Name → DHT Lookup → IPNS Record → Signature Verify → IPFS Path
```

5. API 계층 컴포넌트 (API Layer)

5.1 명령어 컨텍스트 시스템

Context 구조체

```
type Context struct {
ConfigRoot  string
ReqLog *ReqLog
Plugins *loader.PluginLoader
```

```
Gateway  bool

api coreiface.CoreAPI

node *core.IpfsNode

ConstructNode  func() (*core.IpfsNode, error)

}
```

생명주기 관리:

- **지연 초기화**: 필요할 때만 노드 생성
- **리소스 정리**: 요청 완료 후 자동 정리
- **요청 로깅**: 모든 API 요청 추적

5.2 CoreAPI 인터페이스

주요 API 그룹:

- **Unixfs API**: 파일 시스템 작업
- **Block API**: 블록 수준 작업
- **DAG API**: IPLD DAG 작업
- **Name API**: IPNS 작업
- **Pin API**: 핀 관리
- **Pubsub API**: 발행-구독 메시징

6. 컴포넌트 간 상호작용 패턴

6.1 의존성 주입 패턴

IPFS는 Uber FX 프레임워크를 사용하여 의존성 주입을 구현합니다:

```
// 컴포넌트 그룹 정의
var Core = fx.Options(
fx.Provide(Dag), // DAG Service 제공
fx.Provide(FetcherConfig), // Fetcher 설정 제공
fx.Provide(PathResolverConfig), // 경로 해석기 설정 제공
)

// 온라인 모드 컴포넌트
func Online(bcfg *BuildCfg, cfg *config.Config) fx.Option {
return fx.Options(
fx.Provide(Bitswap), // Bitswap 프로토콜
fx.Provide(OnlineExchange), // 온라인 블록 교환
fx.Provide(DNSResolver), // DNS 해석기
fx.Provide(Namesys), // 이름 시스템
LibP2P(bcfg, cfg), // libp2p 네트워크 스택
)
}
```

6.2 생명주기 관리

초기화 순서:

1. **설정 로드**: Repository → Config
2. **신원 설정**: Identity → PrivateKey → PeerID

3. **저장소 초기화**: Datastore → Blockstore
4. **네트워크 설정**: Host → DHT → Bitswap
5. **서비스 시작**: DAG → BlockService → NameSystem
6. **API 활성화**: HTTP API → Gateway

종료 순서:

1. **API 중지**: 새로운 요청 차단
2. **서비스 정리**: 진행 중인 작업 완료
3. **네트워크 종료**: 연결 정리
4. **저장소 동기화**: 데이터 플러시
5. **리소스 해제**: 메모리 및 파일 핸들 정리

6.3 오류 처리 및 복구

계층별 오류 처리:

- **API 계층**: HTTP 상태 코드 및 JSON 오류 응답
- **서비스 계층**: 구조화된 오류 타입 및 컨텍스트
- **네트워크 계층**: 연결 재시도 및 피어 블랙리스트
- **저장소 계층**: 데이터 무결성 검증 및 복구

복구 전략:

- **자동 재시도**: 일시적 네트워크 오류
- **대체 경로**: 피어 또는 게이트웨이 실패 시
- **부분 복구**: 일부 데이터 손실 시 가능한 부분 복구
- **안전 모드**: 심각한 오류 시 읽기 전용 모드

이러한 컴포넌트들의 협력을 통해 IPFS는 확장 가능하고 신뢰할 수 있는 분산 파일 시스템을 구현합니다. 각 컴포넌트의 모듈화된 설계는 시스템의 유지보수성과 확장성을 보장합니다.